

FAST LAB • TAMPERE UNIVERSITY

MIR Suite

Software integration and hardening of the FAST Lab dual-arm mobile manipulator

Project report • Version 2.0 • October 2026

Author	Francisco Javier Martínez Peña Universidad Autónoma del Estado de México (UAEM)
Supervisor	Prof. José Luis Martínez Lastra FAST Lab, Tampere University
Period	June – July 2026
Platform	MiR200 • 2 × UR5e • 2 × BrainCo Revo1 • Orbbec Gemini 335Lg • 2 × Nordbo NRS-6200 • NVIDIA Jetson AGX Orin
Repository	github.com/fjimp-dev/TUNI-PROJECT (release v1.0)
Companion document	MIR Suite — Technical Documentation, v2.0

Executive summary

At the start of this project the FAST Lab dual-arm mobile manipulator could move, but it could only be operated reliably by the person who had programmed it. Every process was started by hand over SSH on the onboard computer, in a specific order; the mobile base republished roughly 150 of its internal topics into the ROS 2 domain shared with the arm controllers; and there was no user interface, no access control and no isolation between components.

Between June and July 2026 I restructured this software into the **MIR Suite**: seven Docker services, one per robot subsystem, operated from a web interface with authentication, role-based permissions and a one-click launcher. The most consequential change was replacing the full MiR-to-ROS 2 republisher with a selective bridge — designed by Wael for a related project and integrated, supervised and hardened here — which reduced the MiR topics entering the shared domain from about 150 to 7. The MiR200 safety stop (Error 9000) that had accompanied the old configuration was not observed again between deployment on 10 July and the end of my stay on 27 July.

The system can now be handed over: a lab member can start it, inspect it, move the arms and view the camera from a browser without terminal knowledge. It is not finished, and this report is deliberately explicit about where it falls short. Section 6 lists the known limitations — most importantly, that the ROS 2 graph is reachable from the whole lab network without authentication, that the software emergency stop does not currently cancel arm motions, and that validation was functional rather than long-duration — and Section 7 lists the open items with suggested owners and priorities.

Table 1. Key outcomes. "Basis" states what each claim rests on.

Area	Before (June 2026)	After (end of July 2026)	Basis
Operation	Processes launched by hand over SSH, in a fixed order	Web interface and one-click launcher; arm driver started deliberately from the interface	Implemented and used in the lab
MiR ↔ ROS 2 traffic	≈150 MiR topics republished into the shared domain	7 topics (6 subscribed, 1 published)	Bridge configuration
MiR Error 9000	Recurring when the base entered Running mode	Not observed, 10–27 July 2026	Operational observation (≈2.5 weeks)
Arm motion reliability	Moves reported as successful while the arm stayed still	Moves of 0.02 rad or more verified against joint states; UR program resent automatically	Implemented; tested on hardware
Camera stream	≈1–2 fps in the browser (observed)	≈30 Hz camera topic at 480 × 270	ros2 topic hz (publication rate)
Isolation	All processes on the host operating system	7 containers, none privileged; resource limits and health checks	docker-compose.yml
Access control	None beyond SSH access to the computer	Login, two roles, per-user control permission, second password for the web terminal	Black-box test, 24 July 2026
Data recording	Unbounded rosbag recording (wrote 522 GB, filled the disk)	10-minute limit, 2 GB split files, raw images excluded	Implemented

1. Context and objectives

1.1 The platform

The platform is a dual-arm mobile manipulator: a MiR200 mobile base (which runs ROS 1 internally) carrying a torso with two Universal Robots UR5e arms, two BrainCo Revo1 hands, an Orbbec Gemini 335Lg RGB-D camera and two Nordbo NRS-6200 force/torque sensors. An NVIDIA Jetson AGX Orin (Advantech MIC-733, hostname "Kevin") running Ubuntu 22.04 and ROS 2 Humble is the onboard computer. A Teltonika RUTX50 5G router provides the robot network (192.168.1.0/24) and its Wi-Fi. The arms are mounted in a reversed orientation on the torso, so joint directions are not always intuitive — a point that matters for anyone commanding them.

1.2 Starting point

The original control software was written by Eemil Rintala (repositories `pbk_system`, `pandai_ark` and a vendored copy of the UR ROS 2 driver). It worked, but had five limitations that made it hard to use and to hand over:

- **Manual operation.** Running the robot meant opening SSH sessions on the Jetson and typing the right commands in the right order.
- **DDS saturation.** The MiR link republished all ≈ 150 topics of the base into the ROS 2 domain used by the arm controllers. This coincided with MiR Error 9000 (safety PLC and emergency stop) on entering Running mode and with missed or delayed arm commands.
- **No isolation.** All processes ran directly on the host, with absolute paths and dependencies that prevented reproducing the environment elsewhere.
- **No interface or visibility.** There was no way to see the robot state, the camera or the logs without the command line.
- **No access control.** Anyone with access to the computer could command the robot.

1.3 Objectives

1. Containerize every component so that it runs reproducibly and in isolation.
2. Reduce the MiR traffic on the shared DDS domain to what the system actually uses.
3. Provide a unified web interface with authentication and access control.
4. Apply least privilege to every component.
5. Document the architecture and the operating procedures so that the system can be handed over.

2. Scope and contributions

Table 2 separates my own work from what I adopted or built upon. Eemil's original projects were not modified: the suite mounts its own copy of the ROS workspace.

Table 2. Contributions.

Component	My contribution	Built on / contributed by
Containerization (7 services, profiles for real arms and simulation)	Design and implementation	—
Web interface (FastAPI backend, Svelte 5 frontend)	Design and implementation	—

Component	My contribution	Built on / contributed by
MiR selective bridge	Integration into the suite; supervision (liveness check, watchdog, reconnection with back-off); diagnosis of the Wi-Fi issue	Bridge design: Wael (fortis-uc1-tau / TK32)
UR5e dual-arm control	Containerized driver; launch patch; motion verification and automatic recovery; start-up checks; payload and freedrive controls	Driver configuration and workspace: Eemil Rintala; Universal Robots ROS 2 driver
Security hardening	Design and implementation	—
Documentation	Technical documentation, operator manual, this report	—

3. Approach

Six principles guided the work: do not modify the original projects; one container per subsystem; least privilege; web-first operation; **nothing moves by default** (the arm driver never starts automatically, not even with the launcher); and document as decisions are made. Two platform constraints shaped the design: the Jetson kernel does not support Docker bridge networks, so every container uses host networking, and FastDDS needs shared memory between processes, so `/dev/shm` is shared across the ROS containers.

Table 3. Timeline, reconstructed from the repository history.

Dates	Work
15–19 Jun	First version of the suite: containerized UR driver, action bridge and a first interface; network topology mapped.
22–29 Jun	Both UR5e arms integrated; MiR200 diagnosed; backend input validation and timeouts; interface rewritten in Svelte with a 3D view of the arms; simulation profile created as a test bench; login added.
3–10 Jul	Repository reorganized by robot component; security hardening (privileged mode removed, authentication and command validation tightened); selective MiR bridge integrated (10 Jul).
13–16 Jul	HTTPS with a private certificate authority; recording bounded after it filled the disk; camera USB recovery; self-healing data path; internal audit (a permission hole closed, error codes corrected); operator manual verified against the running system.
24–27 Jul	Second password for the web terminal; software emergency stop; health checks and resource limits on every service; this report; desktop launcher.

4. Results

4.1 Architecture

Figure 1 shows the resulting system. A reverse proxy (Caddy) is the only web service exposed to the lab network; the backend, rosbridge and the joint server listen only on the Jetson itself. The browser receives the camera stream through rosbridge and sends every command through the authenticated REST API, never through rosbridge. The ROS 2 traffic itself (DDS) is a different matter: because all containers use host networking, the ROS 2 graph is reachable from the lab network (Section 6).

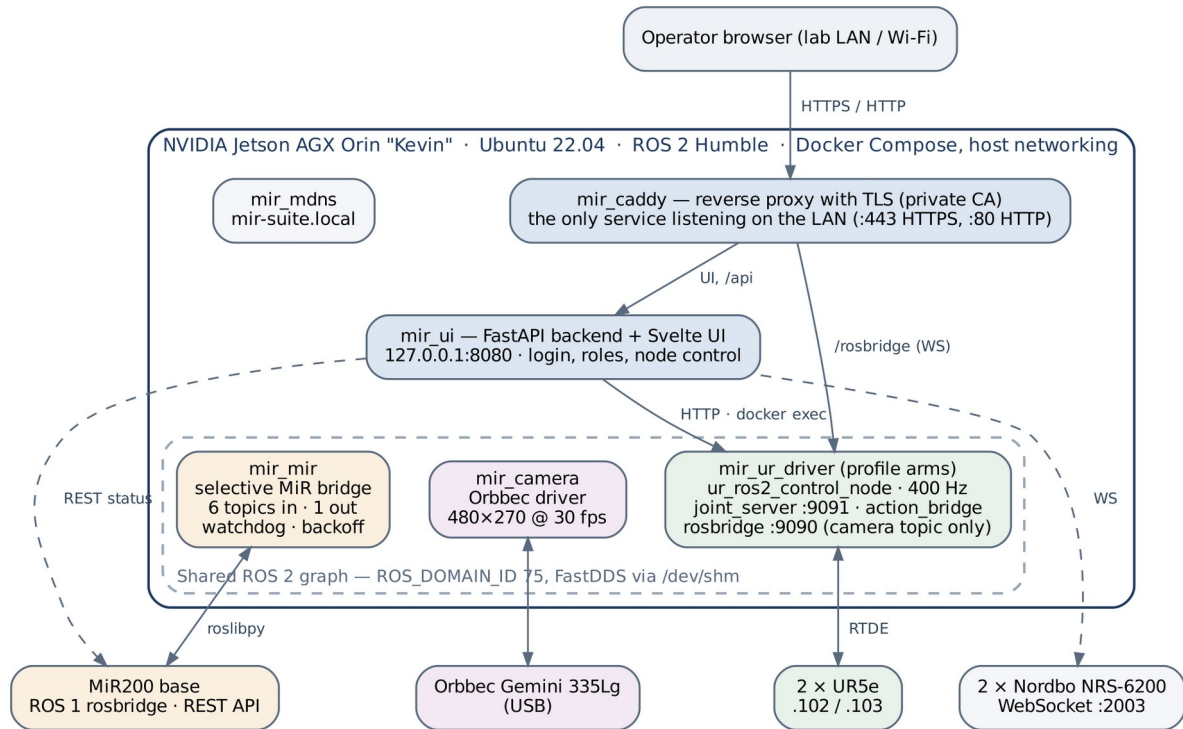


Figure 1. MIR Suite architecture at handover (end of July 2026).

4.2 Mobile base

The selective bridge subscribes only to the front and rear laser scans, odometry, transforms and battery level, and publishes only velocity commands — 7 topics instead of ≈ 150 . It deliberately drops the MiR's map $\rightarrow$ odom transform so that a future on-robot localization system can take that role. I added three supervision layers: a liveness check that records each odometry message, a watchdog that restarts the bridge after 25 s without data (with a 60 s grace period after each start), and a reconnection loop whose wait doubles after each quick failure up to 60 s and resets after a run longer than 60 s. The reduction of about 95 % refers to this **topic count**; DDS bandwidth was not measured before and after.

Three issues with the base remain and are physical or network-level rather than software: the base must be moved back to the 2.4 GHz network where its link was stable; it occasionally stops itself reporting an obstacle that is not there, consistent with a brief interruption of the rear safety scanner (loose cable or dirty window suspected, not yet inspected); and its clock is about ten years off.

4.3 Arms

Both UR5e arms are driven by a single `ur_ros2_control_node` in headless mode, publishing joint states at about 400 Hz. The most frustrating fault of the project was arm moves that the controller reported as successful while the arm did not move. The cause was a two-point trajectory whose start point was a slightly stale joint reading: the controller commanded an instant correction, the resulting velocity spike tripped the robot's safety speed limit, and the protective stop silently halted the external-control program. Moves are now sent as a single target point with a minimum duration of 3 s. After each move of 0.02 rad or more the joint state is checked, and if the arm did not move the backend resends the UR program and retries. Wrist joints can only be moved with an explicit flag. The interface currently offers elbow jogs of ± 0.1 rad per arm; other joints are reachable through the API.

The interface also sets the payload and toggles freedrive (hand-guiding) per arm. Freedrive is refused in the interface until a payload above 1 kg has been set for that arm, so that the arm can hold its own weight once compliant. This check uses the **configured** payload, not a measured one, and is enforced by the interface only (see Section 6).

4.4 Camera

The live view initially reached about 1–2 frames per second. The bottleneck was the encoding of frames for the browser on the Jetson, not the camera. Running the colour stream at 480×270 and sending it compressed brought the end-to-end rate to about 30 Hz. A second, mutually exclusive mode adds depth and a coloured point cloud for perception work, and a "Reset USB" control recovers the camera when it drops off the USB bus, which happened repeatedly.

4.5 Operator interface

The interface has six views — Overview, Arms, MiR, Camera, System and Terminal (Figure 2). Nothing starts on login; nodes are started explicitly from the System view, and each has a short explanation of what it does. A desktop launcher starts or stops the always-on services and opens the interface. Starting the arm driver remains a separate, deliberate step. One dependency is not obvious and should be fixed: `rosbridge` and the joint server run inside the arm container, so the Camera view and the arm controls work only once that container has been created (`docker compose --profile arms up -d`, which does not start the driver itself).

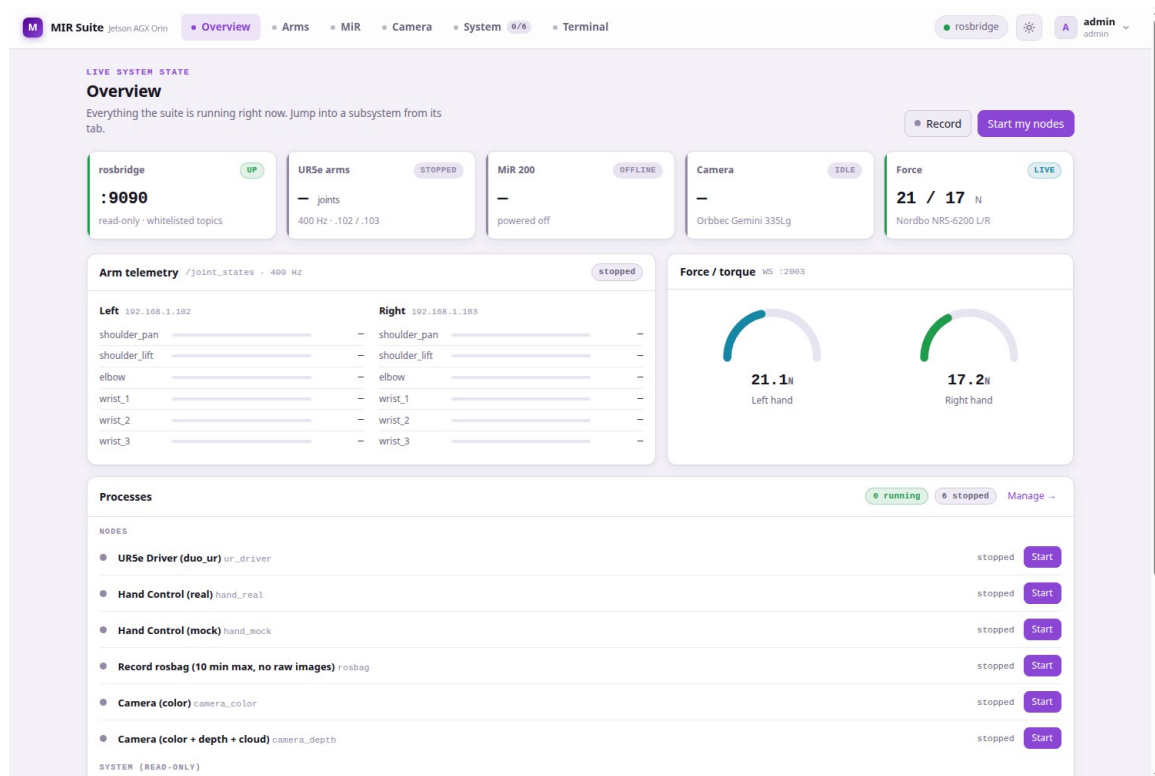


Figure 2. Overview view: service status, arm telemetry, force readings and node control.

4.6 Security

- **Least privilege.** No container runs with `privileged: true`. The arm driver has only the real-time capabilities it needs (`SYS_NICE`, `IPC_LOCK`); device access is granted through cgroup rules for the USB device classes. Every service has memory and CPU limits.
- **Web surface.** Only the reverse proxy is exposed for web traffic; `rosbridge` allows the camera topic only and denies services, actions and parameters, so the arms cannot be commanded through it. (DDS traffic is not covered by this; see Section 6.)
- **Authentication.** Passwords are stored as PBKDF2-SHA256 hashes (200,000 iterations); sessions expire after 24 h; an account locks for 5 minutes after 8 failed logins; two roles (administrator, user) plus a per-user permission to control the robot, enforced by the backend.

- **Web terminal.** Administrators only, with a second, independent password compared in constant time, limited to a whitelist of containers. On 24 July a black-box test confirmed that a missing or invalid token, a non-administrator, a wrong password and a non-whitelisted container are all rejected before a shell opens.
- **Encryption.** HTTPS with a private root certificate authority that signs the server certificate; the CA key is kept only on the Jetson and is not in the repository.

4.7 Reliability and operations

Every service has a health check. The data path from the robot to the browser recovers on its own when `rosbridge` or the camera restarts. Recording is bounded to 10 minutes, split into 2 GB files and excludes raw images, after an unbounded recording wrote 522 GB and filled the Jetson's disk. A simulation profile runs the same arm-driver image against simulated hardware, so that endpoints can be tested without moving the robots.

5. Validation

Validation was functional: each feature was exercised on the real hardware or on the simulation profile until it behaved as intended. Specifically:

- **Hardware tests in the lab:** jog moves of both arms from the interface, payload and freedrive, camera streaming, MiR status and bridge data.
- **Rates** measured with `ros2 topic hz`: joint states ≈ 400 Hz, compressed camera ≈ 30 Hz; about 41 topics in the shared graph. (The MiR odometry rate is recorded inconsistently in the notes — 3.5 Hz and 41 Hz — and should be re-measured.)
- **Security:** the black-box test of the web terminal described in Section 4.6 (24 July 2026).
- **Automated checks:** 24 backend unit tests pass (re-run on 8 October 2026). `run_tests.sh`, which also checks syntax and validates the Compose file, stopped after its first check because of a shell bug; this was found while preparing this report and fixed in release v1.0, where all 19 checks pass.

What this validation does not show

No long-duration test was run. The Error 9000 result rests on about 2.5 weeks of normal lab use, not on a controlled test with the base in Running mode and the arms active.

DDS bandwidth and latency were not measured before and after the bridge change; the reduction is a topic count.

The unit tests exercise copies of backend functions rather than importing the production code, so they do not protect `main.py` against regressions.

The software emergency stop was not tested end-to-end; review of the code for this report showed that it does not work as intended (Section 6).

6. Known limitations and risks

These limitations were identified by reviewing the code against the documentation when preparing this report. None has caused an incident that I know of, but the first three should be addressed before the robot is used with visitors or reachable from outside the lab.

Table 4. Known limitations, ordered by risk.

Limitation	Risk	Recommendation
ROS 2 graph open to the lab network. All containers use host networking with FastDDS in domain 75 and no DDS security.	Any device on the robot network can publish base velocity commands or send arm trajectories without logging in. This bypasses all the web-level access control.	Restrict DDS to the Jetson (shared memory / localhost-only transport or an interface whitelist), or enable SROS2; keep untrusted devices off the robot network.
Software emergency stop does not cancel arm motion. The cancel command is built with incorrect quoting and is rejected; even when fixed, it only cancels goals sent through the action bridge, not jog moves. In practice it only disables freedrive. It also asks for confirmation before acting.	Operators may rely on a button that does not stop the arms.	Fix and test it, remove the confirmation, and label it clearly as a software stop. The hardware emergency stops remain the safety device.
Exposed credentials. The interface password, the router administrator and SSH passwords and the Wi-Fi key were committed to the repository (public) and remain in its history; two notes files still contained some of them until release v1.0.	Anyone can read them.	Change all of them; make the repository private or move it to the lab organization.
Images cannot be rebuilt from the repository alone. Two Dockerfiles reference files that no longer exist, and the UR image's base image (pbd_system) has no recipe in the repository.	The running images on the Jetson are the only working copy.	Fix the Dockerfiles, add a recipe for the UR base image and test a clean build.
Plain HTTP is served on port 80 next to HTTPS.	Passwords and commands travel unencrypted on the lab Wi-Fi.	Install the lab CA on operator machines, then disable port 80.
Possible overshoot on large moves (> 0.25 rad): the move lasts longer than the 15 s wait for its result, and the retry resends the full delta from the partly moved position. Not observed; the interface sends 0.1 rad.	An arm could move past its target.	Scale the result timeout with the move duration; retry only the remaining delta.
The UR start-up script continues if the arms do not confirm their RTDE connection within 45 s.	The interface may show a started driver that cannot move the arms.	Fail the start and report it.
The freedrive payload check exists only in the interface and uses the configured payload.	Can be bypassed through the API; a wrong payload makes a compliant arm drift.	Enforce it in the backend.
The interface backend has the Docker socket.	A compromise of the interface gives root on the Jetson.	Use a socket proxy limited to the calls the backend needs.
The camera stream (/rosbridge) is not behind the login.	Anyone on the lab network can view the camera.	Authenticate /rosbridge at the proxy.
Factory credentials remain on some devices (for example, the MiR web interface); the MiR address default differs between components (.13 / .14).	Unauthorized changes; wrong address after a network change.	Change the credentials; fix the DHCP reservation and align defaults.

Limitation	Risk	Recommendation
Backend in a single file (≈1,700 lines); unit tests do not import it.	Harder to maintain; regressions not caught.	Split by area and test the real modules.

7. Open items and next steps

Table 5. Open items. Owners are suggestions.

Item	Suggested owner	Priority
Change every exposed credential: interface accounts, router administrator and SSH, Wi-Fi key	Lab	High
Restrict the ROS 2 (DDS) traffic to the Jetson	Next developer	High
Fix and test the software emergency stop	Next developer	High
Adjust the UR5e safety speed limit, which occasionally stops normal moves	Wael (robot settings)	High
Move the MiR back to the 2.4 GHz network with a fixed DHCP reservation	Wael / network	High
Disable plain HTTP once the lab certificate is installed on operator machines	Lab	Medium
Make the images rebuildable from the repository (Dockerfiles, base images)	Next developer	Medium
Fix the move timeout for large deltas; fail UR start-up when RTDE does not connect	Next developer	Medium
Inspect the MiR rear safety scanner (cable, window)	Lab technician	Medium
Controlled test: base in Running mode with both arms active, several hours	Next developer	Medium
Remote access over 5G (plan agreed; requires a data-plan change and HTTPS only)	Lab	After the items above
Correct the MiR clock (affects log timestamps only)	Lab	Low
Phase 2: hand battery system, motorized neck, autonomous navigation (SLAM)	Next phase	—

8. Handover

- **Code:** github.com/fjmp-dev/TUNI-PROJECT, release v1.0 (branch main). On the Jetson: `/home/lab/Desktop/MIR/mir_suite`.
- **Documentation:** `documentation/TECHNICAL_DOCUMENTATION.docx` (reference), `documentation/MIR_SUITE_MANUAL.md` (operator manual), `SECURITY.md` (hardening log) and a README in each component folder.
- **Start and stop:** run `docker compose --profile arms up -d` once in the suite folder (it creates all services, including the arm container with rosbridge and the joint server, but does not start the arm driver). Afterwards the desktop launcher starts and stops the suite. For the arms, power them on at their

control panels and start the UR driver from the System view. For tests without hardware use `--profile sim` instead of `--profile arms` — never both, since they share the ROS domain and ports.

- **Secrets:** only in `config/.env` and `UI/backend/data/` on the Jetson; neither is in the repository. `config/env.example` lists every setting without values.
- **Authorship of early commits:** commits from 15 to 29 June 2026 appear under the git identity "EemilR" because they were made from the lab workstation, which had that identity configured. They are part of this project.

9. Acknowledgements

I thank Prof. José Luis Martínez Lastra for hosting me at the FAST Lab; Eemil Rintala, whose original system was the starting point; Wael, whose selective bridge was the key to the MiR integration and who helped with the robots throughout; and the FAST Lab team.

Francisco Javier Martínez Peña · fco_javi_mtz@outlook.com